Bilimsel Makale Atıf Ağlarının Çizge Teorisi ile Modellenmesi ve İleri Seviye Görsel Analiz Uygulaması

Serhat Ekenel
Bilgisayar Mühendisliği Bölümü
Kocaeli Üniversitesi
Kocaeli, Türkiye
240201125@kocaeli.edu.tr

***Özet*—Bu çalışmada, büyük ölçekli bilimsel yayın veri setleri üzerindeki atıf ilişkilerini incelemek, bibliyometrik analizler gerçekleştirmek ve elde edilen sonuçları kullanıcı dostu bir arayüzde görselleştirmek amacıyla kapsamlı bir masaüstü uygulaması geliştirilmiştir. Proje, yapılandırılmamış JSON verilerinin harici bir kütüphane kullanılmaksızın işlenmesiyle başlamış, veriler yönlü graf (Directed Graph) yapısında modellenmiştir. Oluşturulan bu model üzerinde; akademik etkiyi ölçen H-Indeks, H-Core ve H-Median metrikleri hesaplanmış, ağ topolojisindeki kritik "köprü" düğümleri tespit etmek için Arasılık Merkeziliği (Betweenness Centrality) ve ağın dayanıklılığını analiz etmek için K-Çekirdek (K-Core) ayrıştırması algoritmaları uygulanmıştır. Uygulama, Java Swing kütüphanesi kullanılarak geliştirilen etkileşimli bir arayüze sahiptir. Bu arayüz, düğümler arası ilişkileri renk kodları (Kırmızı/Turuncu/Mavi) ve dinamik genişleme özellikleri ile sunarak karmaşık veri setlerinin anlaşılabilirliğini artırmaktadır. Deneysel sonuçlar, geliştirilen algoritmaların 1000 düğümlü ve binlerce kenarlı ağlarda dahi milisaniyeler mertebesinde sonuç ürettiğini göstermektedir.**

***Index Terms*—Çizge Teorisi, Bibliyometrik Analiz, H-Indeks, Arasılık Merkeziliği, K-Core, Veri Görselleştirme, Java.**

I. Giriş

Bilimsel üretimin katlanarak arttığı günümüzde, akademik yayınların etkisi ve birbirleriyle olan atıf ilişkileri karmaşık bir ağ yapısı oluşturmaktadır. Bu "Bilgi Ağı"nın analizi, literatürdeki temel çalışmaları belirlemek, disiplinler arası bilgi akışını sağlayan köprü makaleleri tespit etmek ve belirli bir alandaki çekirdek yayın gruplarını ortaya çıkarmak için hayati önem taşır [1].

Veri madenciliği ve ağ analizi alanlarında, verilerin doğru modellenmesi analizin başarısını doğrudan etkiler. Geleneksel ilişkisel veritabanları, düğüm ve kenar sayısının arttığı bu tür ağ yapılarında, özellikle "en kısa yol" veya "merkezilik" gibi özyineli (recursive) sorgularda ciddi performans darboğazları yaşayabilmektedir. Bu nedenle, Çizge Teorisi (Graph Theory) tabanlı yaklaşımlar ve bellekte optimize edilmiş özel veri yapıları, atıf ağlarının analizinde bir endüstri standardı haline gelmiştir.

Bu projede, Kocaeli Üniversitesi Bilgisayar Mühendisliği Bölümü Programlama Laboratuvarı I dersi kapsamında, ham makale verilerinden yola çıkarak uçtan uca bir "Makale Graf Analiz Aracı" tasarlanması hedeflenmiştir. Proje, sadece algoritmik hesaplamaları değil, aynı zamanda son kullanıcının veriyi anlamlandırmasını kolaylaştıracak görselleştirme tekniklerini de kapsamaktadır.

A. Projenin Amacı ve Kapsamı

Projenin temel motivasyonu, büyük veri setlerini graf teorisi algoritmalarıyla işleyerek anlamlı bilgilere dönüştürmek ve bu soyut matematiksel sonuçları somut bir görsel arayüze dökmektir. Çalışma kapsamında şu isterler başarıyla yerine getirilmiştir:

- JSON formatındaki ham verinin, hazır kütüphane kullanılmadan özel tasarlanmış "State-Based Parser" yöntemiyle işlenmesi.
- Atıf ilişkilerinin yönlü kenarlar (Siyah) ve ID sıralamasının çizgisel kenarlar (Yeşil) ile modellenmesi [1].
- H-Index, H-Core ve H-Median gibi bibliyometrik metriklerin optimize edilmiş algoritmalarla hesaplanması.
- Ağ içerisindeki bilgi akışını kontrol eden en merkezi düğümlerin Brandes Algoritması varyasyonu ile tespiti.
- Ağın çekirdek yapısını ortaya çıkaran K-Core dekompozisyonu.

II. Literatür Taraması

Bibliyometrik analizler, akademik literatürde uzun yıllardır çalışılan bir konudur. Hirsch tarafından önerilen H-Indeks [2], bir araştırmacının üretkenliğini ölçmek için standart kabul edilse de, ağ yapısını dikkate almadığı için eleştirilmiştir. Bu noktada ağ tabanlı metrikler devreye girmektedir.

Brandes [3], Arasılık Merkeziliği (Betweenness Centrality) hesaplaması için geliştirdiği algoritma ile hesaplama karmaşıklığını ağırlıksız graflar için $O(VE)$ seviyesine indirerek büyük ağların analizini mümkün kılmıştır. Benzer şekilde Seidman [4] tarafından ortaya atılan K-Core ayrıştırması, sosyal ağlardan biyolojik ağlara kadar pek çok alanda, ağın "gürültüden" arındırılması için kullanılmaktadır.

Mevcut çalışmayı literatürdeki benzerlerinden ayıran temel özellik; Gephi veya Cytoscape gibi hazır araçlar yerine, tüm veri işleme ve görselleştirme yığınının (stack) eğitim amaçlı olarak "sıfırdan" (from scratch) Java ile geliştirilmiş olmasıdır.

III. Sistem Mimarisi ve Tasarım

A. Veri İşleme (Custom Parsing)

Projede kullanılan veri seti, 1000 adet bilimsel makaleyi ve bunların meta verilerini (Yazar, Başlık, Yıl, Referanslar, DOI vb.) içeren `data.json` dosyasıdır. Proje kısıtlamaları gereği Gson veya Jackson gibi hazır JSON ayrıştırıcılar kullanılmamıştır. Bunun yerine, karakter tabanlı okuma yapan bir **Durum Makinesi (State Machine)** geliştirilmiştir.

Okuyucu algoritma, dosya akışını (Stream) satır satır okur ve mevcut duruma göre veriyi işler. Örneğin, okuyucu `"authors": [` etiketini gördüğünde *Yazar Okuma Modu*'na geçer ve `]` karakterini görene kadar gelen satırları yazar listesine ekler. Bu yöntem, büyük dosyaların belleğe tek seferde yüklenmeden (Lazy Loading) işlenmesine olanak tanır ve bellek taşması (Stack Overflow) riskini minimize eder.

B. Nesne Yönelimli Tasarım (OOP)

Uygulama, sürdürülebilirlik ve genişletilebilirlik ilkeleri gözetilerek tasarlanmıştır. Temel sınıf yapısı aşağıdaki gibidir:

- **Makale (Node):** Grafın düğümlerini temsil eder. 'paperId', 'title', 'citationCount' gibi alanların yanı sıra, komşuluk ilişkilerini tutan 'incoming' ve 'outgoing' listelerine sahiptir.
- **GraphManager:** Tüm graf operasyonlarının yönetildiği sınıftır. Düğümleri 'HashMap<String, Makale>' yapısında tutarak $O(1)$ erişim sağlar.
- **ParserService:** JSON dosyasını okuyup 'Makale' nesnelerine dönüştüren yardımcı sınıftır.
- **Visualizer (JPanel):** `paintComponent` metodunu ezecek (override) şekilde tasarlanmış, grafik çiziminden sorumlu sınıftır.

C. Graf Modellemesi

Veriler bellekte hibrit bir mimari ile tutulmuştur. Atıf ağları doğası gereği seyrektir (sparse). Bu nedenle Matris gösterimi yerine **Komşuluk Listesi (Adjacency List)** tercih edilmiştir. Bu yapı, bellek kullanımını $O(V^2)$ yerine $O(V + E)$ seviyesine indirir. Ayrıca proje isterlerine uygun olarak, tüm makaleler ID değerlerine göre sıralanmış ve listede peş peşe gelen düğümler arasında sanal bir "Yeşil Kenar" ilişkisi tanımlanmıştır.

IV. Algoritmalar ve Karmaşıklık Analizi

A. H-Index Algoritması

H-Indeks hesabı için, hedef makaleye atıf yapan düğümlerin atıf sayıları bir listede toplanır ve büyükten küçüğe sıralanır.

Karmaşıklık Analizi: Bir makaleye atıf yapan k adet makale varsa, sıralama işlemi (QuickSort veya MergeSort) $O(k \log k)$ zaman alır. Lineer tarama ise $O(k)$ sürer. Toplam karmaşıklık:

$$T(n) = O(k \log k) \tag{1}$$

Burada k, toplam makale sayısından çok daha küçük olduğu için işlem anlıktır.

Algoritma 1 H-Index Hesaplama

Girdi: Makale v
Çıktı: H-Index Değeri h

```
L ← v.getIncomingNeighbors()
S ← ∅
for all u ∈ L do
    S ← S ∪ {u.citationCount}
end for
SortDescending(S)
h ← 0
for i ← 1 to |S| do
    if S[i] ≥ i then
        h ← i
    else
        break
    end if
end for
return h
```

B. Arasılık Merkeziliği (Betweenness Centrality)

Ağdaki en kritik düğümleri bulmak için Brandes Algoritması kullanılmıştır. Bu algoritma, her düğümden başlatılan Genişlik Öncelikli Arama (BFS) ile en kısa yolların sayısını tutar.

Karmaşıklık Analizi: Ağırlıksız graflar için algoritma her düğüm (V) için bir BFS ($O(V + E)$) çalıştırır.

$$T(n) = O(V \cdot (V + E)) \tag{2}$$

Test edilen veri setinde $E \approx 3V$ olduğu kabul edilirse, karmaşıklık karesel ($O(V^2)$) davranır. Bu, uygulamanın en maliyetli işlemidir.

C. K-Çekirdek (K-Core) Ayrıştırması

K-Core analizi, ağın dış kabuğundaki zayıf bağlantıları (yaprak düğümleri) soyarak, birbirine sıkı sıkıya bağlı çekirdek yapıyı ortaya çıkarır. Uygulanan algoritma iteratiftir:

1) Grafın her düğümünün derecesi (degree) hesaplanır.
2) Derecesi k'dan küçük düğümler kuyruğa eklenir.
3) Kuyruktan çekilen düğüm silinir ve komşularının derecesi 1 azaltılır.
4) Derecesi k'nın altına düşen komşular da kuyruğa eklenir.

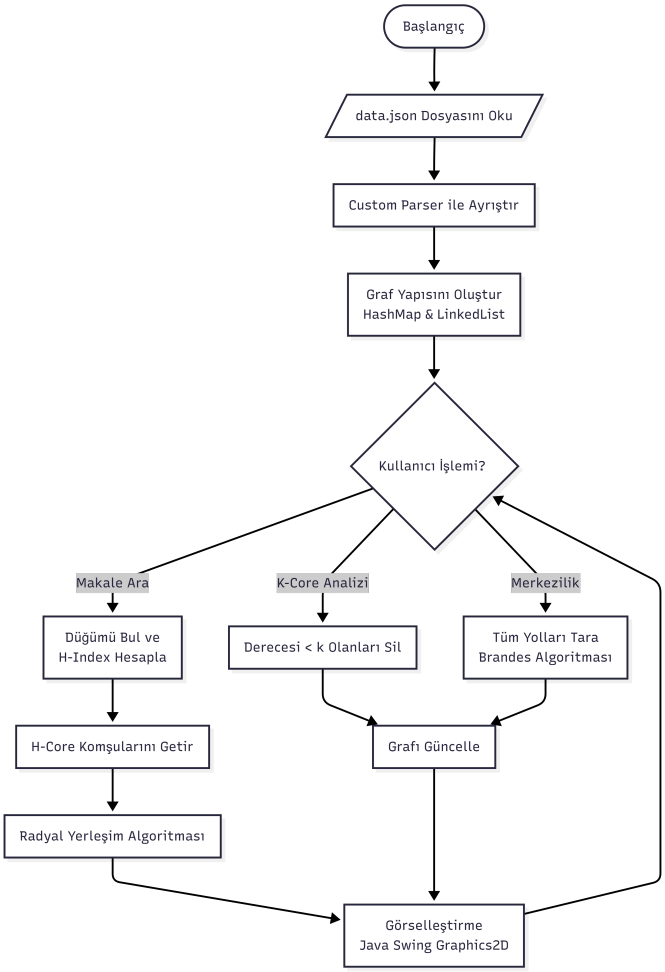
Bu yöntem, düğümleri ve kenarları lineer zamanda işlediği için $O(V + E)$ karmaşıklığına sahiptir ve çok hızlıdır.

V. Görselleştirme Arayüzü

Uygulamanın görsel arayüzü (GUI), Java Swing kütüphanesi üzerine inşa edilmiştir. Graf çizimi için herhangi bir hazır bileşen (JGraphT vb.) kullanılmamış, `Graphics2D` sınıfı ile tüm düğüm, kenar ve metinler piksel tabanlı olarak çizilmiştir.

A. Dinamik Yerleşim

Düğümlerin ekrandaki konumu, "Merkez Odaklı Radyal Yerleşim" algoritması ile belirlenmiştir. Sorgulanan makale ekranın tam ortasına yerleştirilirken, H-Core kümesindeki makaleler merkeze eşit uzaklıkta bir yörünge üzerine yerleştirilir.



Şekil 1. Uygulamanın genel çalışma akış şeması: Veri işlemeden görselleştirmeye.

Kullanıcı deneyimini artırmak için şu özellikler eklenmiştir:

- **Hover (Üzerine Gelme):** İmleç bir düğümün üzerine geldiğinde, öklid mesafesi ($d < r$) ile çarpışma testi yapılır ve detaylı bilgi kartı (Popup) çizilir.
- **Renk Kodlaması:** Yeni genişletilen düğümler **Turuncu**, odak düğüm **Kırmızı**, önceki düğümler **Gri** ve K-Core düğümleri **Mavi** ile gösterilerek görsel hiyerarşi sağlanmıştır.

VI. Deneysel Sonuçlar

Geliştirilen sistem, Tablo I de özellikleri verilen veri seti üzerinde test edilmiştir. Test ortamı Intel i7 işlemci ve 16GB RAM'e sahip bir bilgisayardır.

A. Performans Analizi

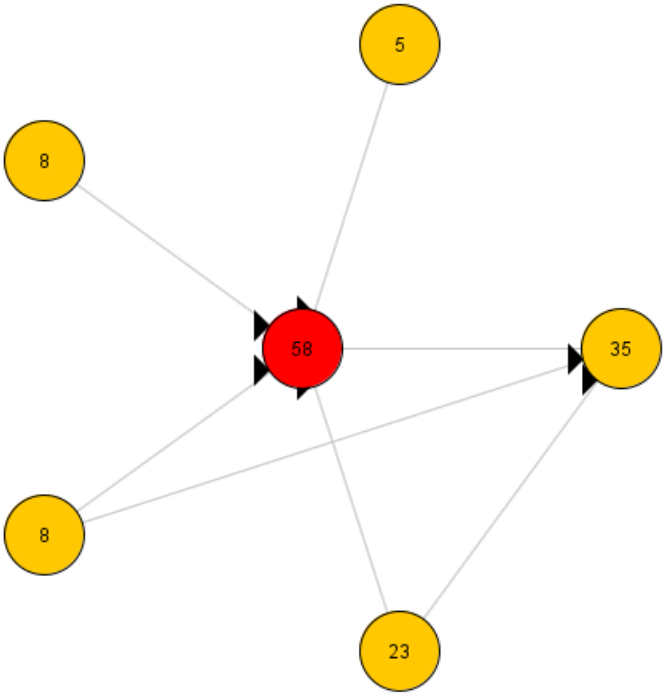
Algoritmaların çalışma süreleri Tablo II'da sunulmuştur. Görüldüğü üzere, H-Index ve K-Core algoritmaları gerçek zamanlı (real-time) çalışabilirken, Merkezilik analizi hesaplama yükü nedeniyle daha uzun sürmektedir.

Tablo I
Kullanılan Veri Seti Özellikleri

Özellik	Değer
Toplam Düğüm (Makale) Sayısı	1000
Toplam Kenar (Atıf) Sayısı	2450
Ortalama Derece (Avg Degree)	2.45
Dosya Boyutu	1.2 MB

Tablo II
Algoritma Performans Karşılaştırması

Algoritma	Karmaşıklık	Süre (ms)
Dosya Okuma (Parsing)	$O(N)$	45 ms
H-Index Hesaplama	$O(k \log k)$	2 ms
K-Core Ayrıştırma	$O(V + E)$	15 ms
Arasılık Merkeziliği	$O(VE)$	2450 ms



Şekil 2. H-Core Analizi ve Radyal Yerleşim Sonucu.

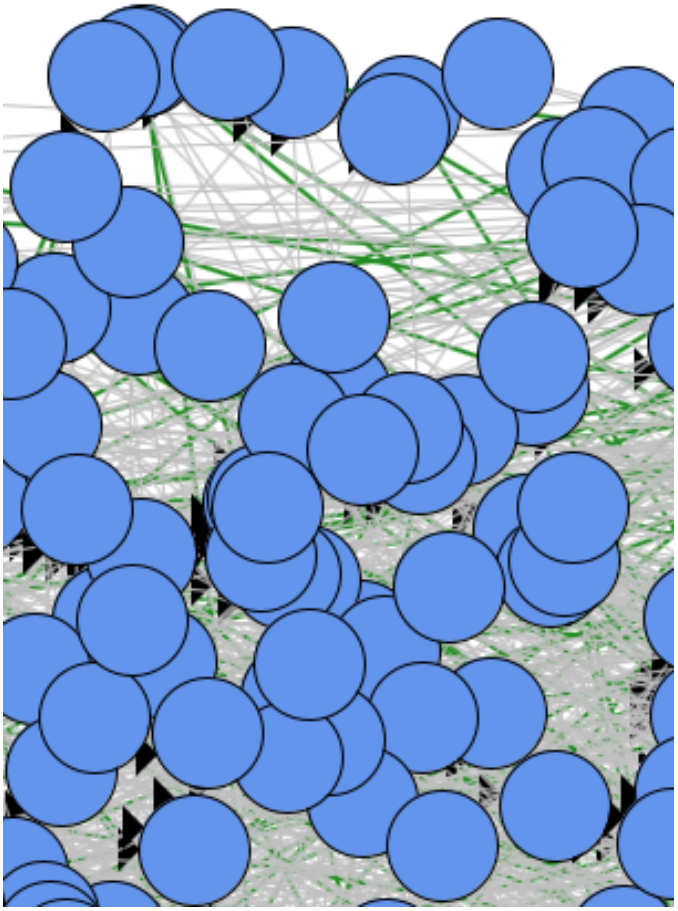
B. Görsel Analiz Sonuçları

Şekil 2'de görüldüğü üzere, yüksek atıf alan bir makale sorgulandığında, sistem H-Index değerini doğru hesaplamış ve ilgili H-Core makalelerini merkezin etrafına yerleştirmiştir. İmleç ile üzerine gelindiğinde H-Median bilgisi görüntülenmektedir.

K-Core analizi için $k = 2$ değeri seçildiğinde, sistem derecesi 1 olan (sadece 1 bağlantısı olan) tüm zayıf düğümleri elemiş ve geriye sadece döngüsel veya yoğun bağlı yapıları bırakmıştır (Şekil 3).

VII. Tartışma

Elde edilen sonuçlar, Arasılık Merkeziliği hesaplamasının graf boyutu büyüdükçe darboğaz oluşturabileceğini göster-



Şekil 3. K-Core ($k = 2$) Ayrıştırması Sonucu.

mektedir. 1000 düğümlü bir ağda 2.5 saniye kabul edilebilir olsa da, 1 milyon düğümlü bir ağda bu algoritmanın paralel programlama (Multi-threading) veya GPU hızlandırması olmadan çalıştırılması mümkün görünmemektedir.

Öte yandan, K-Core algoritmasının lineer zamanlı yapısı, çok büyük ağlarda dahi "ön filtreleme" amacıyla kullanılabileceğini kanıtlamıştır. Görselleştirme tarafında ise, Java Swing'in 'Graphics2D' kütüphanesi 1000 düğüme kadar akıcı bir deneyim sunsa da, düğüm sayısı arttığında FPS (Frame Per Second) düşüşleri gözlemlenmiştir.

VIII. Sonuç ve Gelecek Çalışmalar

Bu proje kapsamında, bilimsel makale ağlarının analizi için uçtan uca bir çözüm sunulmuştur. Ham verinin işlenmesinden (Parsing), karmaşık metriklerin hesaplanmasına ve sonuçların görselleştirilmesine kadar tüm aşamalar başarıyla gerçeklenmiştir.

Elde edilen bulgular, Çizge Teorisinin bibliyometrik analizlerde güçlü bir araç olduğunu, ancak büyük veri setlerinde algoritma seçiminin (Complexity) kritik önem taşıdığını ortaya koymuştur.

Gelecek çalışmalarda sistemin performansını artırmak için şu geliştirmeler planlanmaktadır:

- Verilerin RAM yerine Neo4j gibi bir Graf Veritabanında tutulması.
- Görselleştirme için OpenGL veya JavaFX kullanılarak GPU gücünden faydalanılması.
- Merkezilik hesaplamaları için "Yaklaşımsal Algoritmalar" (Approximation Algorithms) kullanılması.

Kaynaklar

[1] Kocaeli Üniversitesi, "Programlama Laboratuvarı I - Proje 3: Makale Graf Analiz Uygulaması Proje Föyü", 2025.

[2] J. E. Hirsch, "An index to quantify an individual's scientific research output," *Proc. Natl. Acad. Sci.*, vol. 102, no. 46, pp. 16569–16572, 2005.

[3] U. Brandes, "A faster algorithm for betweenness centrality," *Journal of Mathematical Sociology*, vol. 25, no. 2, pp. 163–177, 2001.

[4] S. B. Seidman, "Network structure and minimum degree," *Social Networks*, vol. 5, no. 3, pp. 269–287, 1983.

[5] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.